Пермский национальный исследовательский политехнический университет

Отчет по дисциплинам:

“Информатика”

“Основы алгоритмизации и программирования”

“Введение в инженерную деятельность”

Вариант 20

Выполнил:

студент группы РИС-26-3б

Салищев Данил Романович

Проверил:

Доцент кафедры ИТАС

О.А. Полякова

2026

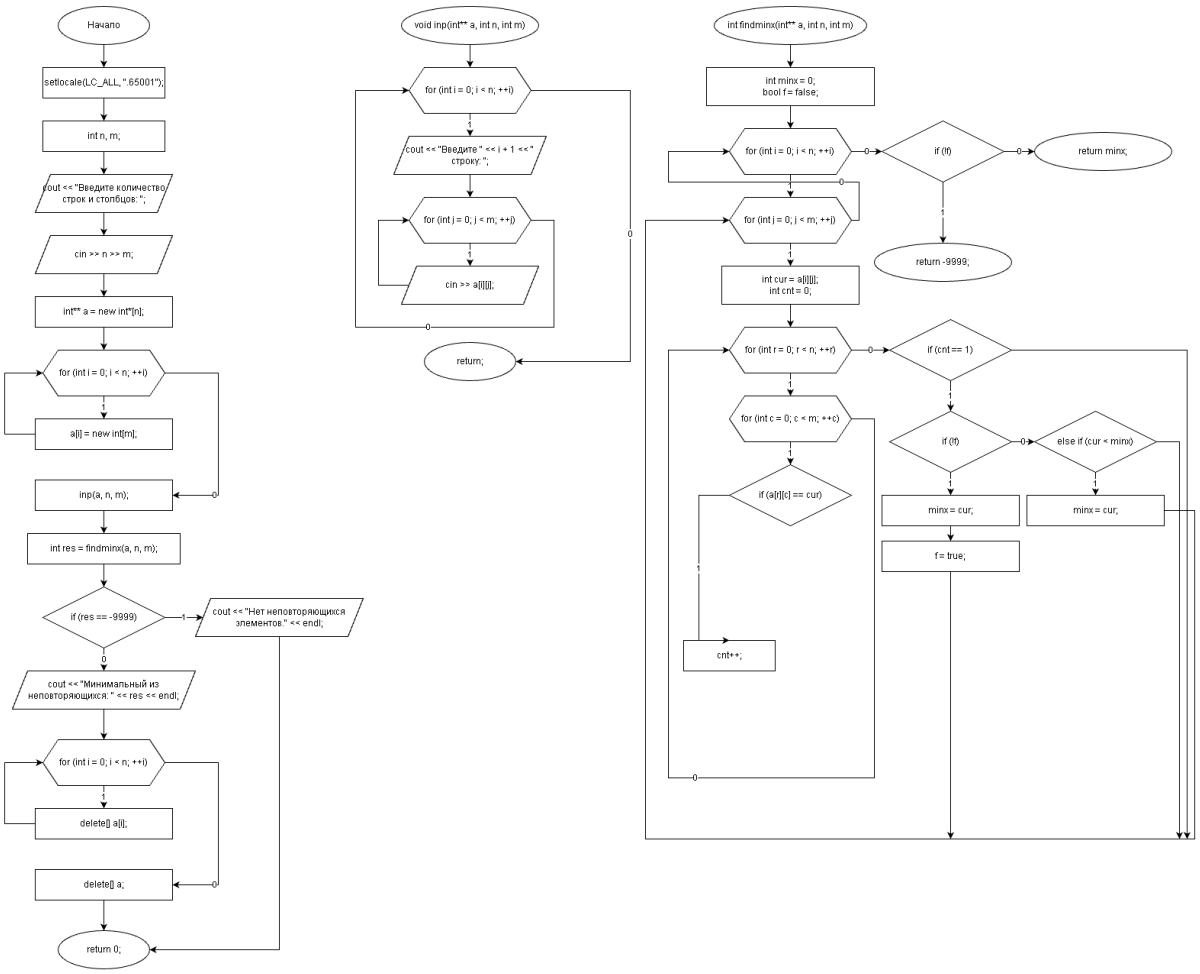
1. Постановка задачи

Найти минимальный из неповторяющихся элементов двумерного массива.

2. Анализ решения

1. **Динамическая память.** Матрица создается через new int*[n] и new int[m], так как размеры n и m становятся известны только после ввода пользователя. Это позволяет не задавать фиксированный размер заранее и не тратить память зря.
2. **Использование функций.** По условию лабы массив передается в функцию как параметр. Ввод вынесен в void inp, поиск результата — в int findminx. Массив передается по указателю (int**), поэтому функция inp заполняет его напрямую, и изменения видны в main. Возвращать массив через return не нужно.
3. **Поиск уникальных элементов.** Для каждого элемента cur запускаются два дополнительных цикла (r, c), которые считают, сколько раз этот элемент встречается во всей матрице. Счетчик хранится в cnt. Если после полного прохода cnt == 1, значит элемент неповторяющийся.
4. **Поиск минимума без** <climits>**.** Чтобы не использовать максимальное число для начального значения минимума, введен флаг f. Пока флаг false, первый найденный уникальный элемент просто записывается в minx. Для всех последующих работает проверка cur < minx. Если уникальных элементов нет вообще, функция возвращает -9999, и main выводит соответствующее сообщение.
5. **Очистка памяти.** Так как использовался new, в конце программы память освобождается через delete[] (сначала для каждой строки, потом для массива указателей). Это нужно, чтобы не было утечек памяти.
6. **Сложность.** По времени — $O(n^2 \cdot m^2)$, так как каждый элемент матрицы сравнивается со всеми остальными (четыре вложенных цикла). По памяти — $O(n \cdot m)$ под саму матрицу.

3. Блок-схема



4. Код

```cpp
#include <iostream>
#include <clocale>
using namespace std;

// Функция ввода матрицы
void inp(int** a, int n, int m) {
    for (int i = 0; i < n; ++i) {
        cout << "Введите " << i + 1 << " строку: ";
        for (int j = 0; j < m; ++j) {
            cin >> a[i][j];
        }
    }
}

// Функция поиска минимального из неповторяющихся
int findminx(int** a, int n, int m) {
    int minx = 0;
    bool f = false;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            int cur = a[i][j];
            int cnt = 0;

            // Считаем, сколько раз cur встречается в матрице
            for (int r = 0; r < n; ++r) {
                for (int c = 0; c < m; ++c) {
                    if (a[r][c] == cur) {
                        cnt++;
                    }
                }
            }

            // Если элемент уникален
            if (cnt == 1) {
                if (!f) {
                    minx = cur;
                    f = true;
                } else if (cur < minx) {
                    minx = cur;
                }
            }
        }
    }

    if (!f) return -9999; // Если уникальных нет
    return minx;
}
```

```cpp
int main() {
    setlocale(LC_ALL, ".65001");
    int n, m;

    cout << "Введите количество строк и столбцов: ";
    cin >> n >> m;

    int** a = new int*[n];
    for (int i = 0; i < n; ++i) {
        a[i] = new int[m];
    }

    inp(a, n, m);
    int res = findminx(a, n, m);

    if (res == -9999) {
        cout << "Нет неповторяющихся элементов." << endl;
    } else {
        cout << "Минимальный из неповторяющихся: " << res << endl;
    }

    for (int i = 0; i < n; ++i) {
        delete[] a[i];
    }
    delete[] a;

    return 0;
}
```

5.Скриншоты

```
Введите количество строк и столбцов: 3 2
Введите 1 строку: 1 2
Введите 2 строку: 0 0
Введите 3 строку: 1 3
Минимальный из неповторяющихся: 2
```

6.Выводы

В ходе выполнения лабораторной работы №5 я научился оформлять программу в виде отдельных функций и передавать в них массивы в качестве параметров. Я выяснил, как работает динамическое выделение

памяти с помощью new и delete и зачем нужно освобождать память в конце программы. Я разобрался, как с помощью вложенных циклов и счетчиков искать неповторяющиеся элементы в матрице и находить среди них минимальный